

SnowPro Specialty: Native Apps — Prep Guide

⚠️ **Verify against the official exam guide as of July 2026** — Snowflake revises exam codes, domains, and weightings periodically. Confirm current details at snowflake.com/certifications before your exam.

PJ's Academy · Prove you can build and distribute apps on the Snowflake Native App Framework.

The Native Apps Specialty exam validates that you can build, package, secure, and monetize applications that run **inside a consumer's Snowflake account** using the Native App Framework and distribute them via the Snowflake Marketplace.

Prerequisite: SnowPro Core certified.

Exam At A Glance

Item	Detail
Exam code	NAS-C01 <i>(verify current code)</i>
Questions	~55
Duration	85 minutes
Cost	\$225 USD
Prerequisite	SnowPro Core

Domain Weightings *(verify against official guide)*

#	Domain	Weight
1	Native App Framework Fundamentals	25%
2	Building & Packaging Applications	30%
3	Security, Data Access & Permissions	25%
4	Distribution, Monetization & Lifecycle	20%

Domain 1 — Native App Framework Fundamentals (25%)

The core idea

A Native App runs **inside the consumer's account** — the provider's code and logic execute on the consumer's data, without the data ever leaving the consumer's account. This is the key differentiator vs a SaaS app.

Key objects

- **Application Package** — the container the provider builds; holds versions, the manifest, and shared content.
- **Manifest** (`manifest.yml`) — declares version, artifacts, privileges required, and setup script.
- **Setup script** (`setup_script.sql`) — runs in the consumer's account to create the app's objects (schemas, procedures, UDFs, Streamlit, roles).
- **Application object** — the installed instance in the consumer's account.
- **Versions & patches** — providers release versioned updates; consumers upgrade.

```
# manifest.yml (simplified)
manifest_version: 1
version:
  name: v1_0
  label: "Version 1.0"
artifacts:
  setup_script: setup_script.sql
  readme: README.md
  default_streamlit: ui.dashboard
privileges:
  - EXECUTE TASK:
      description: "Run scheduled refresh tasks"
```

Domain 2 — Building & Packaging (30%)

Provider workflow

1. Create an **application package**: `CREATE APPLICATION PACKAGE my_app;`
2. Upload artifacts (manifest, setup script, Python/SQL, Streamlit) to a **named stage**.
3. Add a **version**: `ALTER APPLICATION PACKAGE my_app ADD VERSION v1 USING '@stage/path';`
4. Test-install locally: `CREATE APPLICATION my_app_test FROM APPLICATION PACKAGE my_app USING VERSION v1;`

Inside the setup script

- Create versioned schemas, **stored procedures** (Snowpark Python/SQL), **UDFs**, and **Streamlit** apps.
- Define **application roles** (`CREATE APPLICATION ROLE app_user;`) and grant privileges on app objects to them.
- Use **references** to request access to consumer objects (tables the app needs).

```
-- setup_script.sql (runs in consumer account)
CREATE APPLICATION ROLE IF NOT EXISTS app_user;
CREATE OR ALTER VERSIONED SCHEMA core;
GRANT USAGE ON SCHEMA core TO APPLICATION ROLE app_user;

CREATE PROCEDURE core.run_analysis()
  RETURNS STRING LANGUAGE SQL AS $$ ... $$;
GRANT USAGE ON PROCEDURE core.run_analysis() TO APPLICATION ROLE app_user;

-- Streamlit UI shipped with the app
CREATE STREAMLIT core.dashboard FROM '/streamlit' MAIN_FILE='app.py';
GRANT USAGE ON STREAMLIT core.dashboard TO APPLICATION ROLE app_user;
```

Sharing provider data

- Share provider-owned data into the app via the application package (like a share), so the app can join it with consumer data.



Domain 3 — Security, Data Access & Permissions (25%)

The permission model (heavily tested)

- Apps run with **restricted privileges** — they can't silently touch consumer data.
- **References** — the app declares it needs a table/view; the consumer explicitly **binds** their object to the reference (consent).
- **Requested privileges** — declared in the manifest (e.g., `CREATE TASK`, `EXECUTE TASK`, `IMPORTED PRIVILEGES ON SNOWFLAKE DB`); consumer grants them at install/runtime.
- **Application roles** vs consumer roles — app roles gate access to the app's own objects.

```
-- Consumer binds their table to a reference the app requested
CALL my_app.core.register_reference('orders_table', 'ADD',
    SYSTEM$REFERENCE('TABLE','consumer_db.public.orders','PERSISTENT','SELECT'));
```

Data protection

- Provider code is **not visible** to the consumer (logic stays proprietary).
- Consumer data **never leaves** the consumer account.
- Masking/row policies still apply; apps honor the consumer's governance.



Domain 4 — Distribution, Monetization & Lifecycle (20%)

- **Listings** — publish the app to the **Snowflake Marketplace** (public) or a private listing / Data Exchange.
- **Monetization** — free, paid (flat/usage-based), or trial pricing via Marketplace billing.
- **Versioning & upgrades** — release directives (default version, patches); consumers upgrade with control.
- **Release channels** — testing vs production distribution.
- **Telemetry & logging** — event tables for app logs/metrics (with consumer consent).
- **Security review** — Snowflake reviews apps before public Marketplace listing.



High-Yield Facts

- A Native App runs **in the consumer's account** — consumer data never leaves; provider logic stays hidden.
- **Application Package** = provider's build container; **Application object** = installed instance.
- **manifest.yml** declares version, artifacts, and **requested privileges**.
- **References** let the app request consumer objects; the **consumer must bind/consent**.
- **Application roles** gate access to the app's own objects (not consumer roles).
- **Versioned schemas** (`CREATE OR ALTER VERSIONED SCHEMA`) hold app logic per version.
- Distribution & monetization happen via **Marketplace listings**; public listings need a **security review**.
- Ship a **Streamlit** UI inside the app for a full front end.



Practice Questions (10)

Q1. A Native App executes: A) On Snowflake's servers B) In the consumer's account C) On the provider's account D) In the cloud provider VPC

Q2. The provider's build container is the: A) Application object B) Application package C) Stage D) Share

- Q3.** Which file declares version, artifacts, and requested privileges? A) setup_script.sql B) manifest.yml C) app.py D) readme.md
- Q4.** For an app to access a consumer's table, the consumer must: A) Nothing — apps have full access B) Bind their object to a reference (consent) C) Drop RBAC D) Share the account
- Q5.** Access to the app's own objects is gated by: A) Consumer roles B) Application roles C) ACCOUNTADMIN D) PUBLIC
- Q6.** Provider proprietary logic is: A) Fully visible to consumers B) Hidden from consumers C) Public on Marketplace D) Emailed to consumers
- Q7.** Native Apps are distributed via: A) GitHub B) Marketplace listings C) Email D) Snowpipe
- Q8.** Where do you create app schemas, procedures, and Streamlit on install? A) manifest.yml B) setup_script.sql C) The share D) A task
- Q9.** Public Marketplace listings require: A) Nothing B) A Snowflake security review C) ACCOUNTADMIN of the consumer D) A reader account
- Q10.** Consumer data in a Native App: A) Is copied to the provider B) Never leaves the consumer account C) Goes to the Marketplace D) Is public

✓ **Answers**

1-B · 2-B · 3-B · 4-B · 5-B · 6-B · 7-B · 8-B · 9-B · 10-B



2-Week Plan

- **Week 1:** Framework fundamentals + build/package an app (manifest, setup script, Streamlit).
- **Week 2:** Security/references + distribution/monetization + practice. Build one real app end-to-end.

❄️ *Snowflake Mastery — PJ's Academy*